Azentio.ai Backend — Updated Final Documentation

Project: Azentio.ai Intelligent Collaboration Portal

Base URL: `http://127.0.0.1:8000`

API prefix: `/api/v1`

Swagger UI: `http://127.0.0.1:8000/docs`

ReDoc: `http://127.0.0.1:8000/redoc`

OpenAPI: `http://127.0.0.1:8000/openapi.json`

1. Implementation Summary

The backend uses FastAPI, SQLAlchemy 2.0 async, MySQL 8.0, asyncmy, Alembic, Pydantic v2, Argon2 password hashing, and JWT access/refresh tokens.

The implementation is complete through the following areas:

- Authentication, public registration, current user, refresh tokens
- Products and demo enquiries
- Private observations and explicit consent
- Consent withdrawal and shared-learning unpublishing
- Moderation, redaction, publication, entitlements, and product improvements
- Append-only audit events
- Administrator-managed users, memberships, account status, and role assignment
- Active-server-side membership authorization and tenant isolation

No database schema migration was required for public registration or administrator APIs. Alembic is at revision **`phase4_platform_workflows`**, and **`alembic check`** reported no missing upgrade operations.

2. Authentication and Registration

Protected endpoints require:

```
Authorization: Bearer <access_token>
```

Access tokens authenticate a user by UUID. The server reloads the user and all memberships from the database; roles are never accepted from JWT claims or public request bodies. An inactive account is rejected.

Public registration

`POST /api/v1/auth/register`

Authentication: not required.

Accepted fields only:

- **`full_name`**

• **`email`**

• **`password`**

Unknown fields, including **`role`**, **`organisation_id`**, **`is_active`**, **`is_verified`**, and **`hashed_password`**, are rejected by Pydantic with HTTP 422.

Example request:

```
{
  "full_name": "Registered User",
  "email": "registered@example.com",
  "password": "SecurePass123!"
}
```

Example success response:

```
{
  "success": true,
  "message": "Registration successful",
  "data": {
    "id": "123e4567-e89b-12d3-a456-426614174000",
    "email": "registered@example.com",
    "full_name": "Registered User",
    "is_active": true,
    "is_verified": false,
    "created_at": "2026-10-30T12:00:00Z",
    "memberships": []
  },
  "meta": {}
}
```

Status codes:

• **`201 Created`**: registration succeeded

• **`409 Conflict`**: email already exists

• **`422 Unprocessable Entity`**: invalid or extra fields

Passwords are hashed with the existing Argon2 implementation. A registered user starts with no organisation membership and has no organisation-specific access until an administrator assigns an active membership.

Login and refresh

`POST /api/v1/auth/login` accepts **`email`** and **`password`**, returning an access token and refresh token.

`POST /api/v1/auth/refresh` accepts a refresh token and issues new tokens if the account is active.

`GET /api/v1/auth/me` returns the current user and active/inactive membership details.

Invalid, expired, wrong-type, and inactive-user authentication is rejected.

3. Administration API

All administration endpoints require an active membership with the **`admin`** role.

List users

`GET /api/v1/admin/users?page=1&page_size=20`

Returns registered users with memberships.

Get user

GET /api/v1/admin/users/{user_id}

Returns the user and membership records.

Assign membership

POST /api/v1/admin/users/{user_id}/memberships

Example:

```
{
  "organisation_id": "123e4567-e89b-12d3-a456-426614174001",
  "role": "client"
}
```

Permitted role values are:

- **client**
- **moderator**
- **product_owner**
- **employee**
- **admin**

The organisation must exist and be active. Duplicate **(user_id, organisation_id)** memberships return 409. Invalid roles return 422.

Update membership

PATCH /api/v1/admin/users/{user_id}/memberships/{membership_id}

Example:

```
{
  "role": "moderator",
  "is_active": true
}
```

At least one of **role** or **is_active** is required. Unknown fields are rejected.

Revoke membership

DELETE /api/v1/admin/users/{user_id}/memberships/{membership_id}

Revocation is implemented as a soft revocation: **is_active** becomes false. The membership record remains available for auditability and can be reactivated by an administrator.

Activate or deactivate account

PATCH /api/v1/admin/users/{user_id}/status

Example:

```
{
  "is_active": false
```

```
}
```

Administrative status codes:

- **`401 Unauthorized`**: missing/invalid bearer token
- **`403 Forbidden`**: authenticated user is not an active administrator
- **`404 Not Found`**: user, organisation, or membership not found
- **`409 Conflict`**: duplicate membership
- **`422 Unprocessable Entity`**: invalid UUID, role, status body, or missing update field

4. Manual Initial Administrator Provisioning

The application has no initial-admin bootstrap command and automatically creates no administrator.

The initial administrator must be created manually in MySQL by the responsible operator. A compatible setup requires:

1. An existing administrator account in **`users`**, with an Argon2 hash stored in **`hashed_password`**.
2. An existing organisation in **`organisations`**.
3. An active **`organisation_memberships`** row with **`role = 'admin'`**.

No SQL was executed by this implementation. Example template only:

```
INSERT INTO users (
    id, email, full_name, hashed_password, is_active, is_verified, created_at, updated_at
)
VALUES (
    UUID(),
    'admin@example.com',
    'Initial Administrator',
    '<ARGON2_HASH_CREATED_OUTSIDE_THIS_APPLICATION>',
    TRUE,
    TRUE,
    UTC_TIMESTAMP(),
    UTC_TIMESTAMP()
);
```

Then, using the generated user UUID and organisation UUID:

```
INSERT INTO organisation_memberships (
    id, user_id, organisation_id, role, is_active, joined_at
)
VALUES (
    UUID(),
    '<ADMIN_USER_UUID>',
    '<EXISTING_ORGANISATION_UUID>',
    'admin',
    TRUE,
    UTC_TIMESTAMP()
);
```

Generate using the application's existing Argon2 password-hashing function or an equivalent administrator provisioning process. Never store a plaintext password. Use the MySQL column type and UUID format compatible with the existing schema.

5. Database Schema

The existing registration flow requires no schema change:

Table	Registration/admin relevance
users	Public registration creates a user with no memberships
organisations	Membership assignment validates an existing organisation
organisation_memberships	Unique **(user_id, organisation_id)** membership constraint
observations	Organisation-private observations
moderation_reviews	Moderation decisions and redaction
shared_learning_entries	Published redacted learning
organisation_product_entitlements	Organisation entitlement to product learning
product_improvements	Improvement records and status
product_improvement_evidence	Evidence links
audit_events	Append-only administrative and workflow audit data

The ID columns use **CHAR(36)**. SQLAlchemy GUID compatibility is implemented by **app/db/types.py**.

6. Authorization and Isolation Rules

• Public registration cannot select a role or organisation.

• Registration creates **is_active=true** and **is_verified=false** by server policy.

• Organisation-specific access requires an active matching membership.

• A JWT does not grant organisation access or roles by itself.

• **admin** API access requires an active administrator membership.

• Membership assignment and account changes are audited.

• Duplicate memberships are rejected.

• Invalid role values are rejected.

• Revocation is a status change, not hard deletion, preserving the relationship and audit trail.

• Observation private text and another organisation's resources remain hidden by tenant-filtered queries.

• Audit records have no public mutation or deletion endpoint.

7. Implemented API Endpoints

The live OpenAPI schema contains 27 paths and 36 operations.

Area	HTTP Method	Path
Health	GET	**/api/v1/health**
Registration	POST	**/api/v1/auth/register**
Authentication	POST	**/api/v1/auth/login**
Authentication	POST	**/api/v1/auth/refresh**
Authentication	GET	**/api/v1/auth/me**
Products	GET, POST	**/api/v1/products**
Products	GET, PATCH, DELETE	**/api/v1/products/{product_id}**
Demo enquiries	POST, GET	**/api/v1/demo-enquiries**
Demo enquiries	GET	**/api/v1/demo-enquiries/{enquiry_id}**
Demo enquiries	PATCH	**/api/v1/demo-enquiries/{enquiry_id}/status**

Area	HTTP Method	Path
Observations	POST, GET	`/api/v1/observations`
Observations	GET	`/api/v1/observations/{observation_id}`
Consent	POST, DELETE	`/api/v1/observations/{observation_id}/consent`
Moderation	GET	`/api/v1/moderation/observations`
Moderation	POST	`/api/v1/moderation/observations/{observation_id}/decision`
Moderation	POST	`/api/v1/moderation/observations/{observation_id}/publish`
Entitlements	POST	`/api/v1/entitlements/product`
Shared learning	GET	`/api/v1/shared-learning`
Shared learning	GET, DELETE	`/api/v1/shared-learning/{entry_id}`
Improvements	POST, GET	`/api/v1/product-improvements`
Improvements	PATCH	`/api/v1/product-improvements/{improvement_id}/status`
Audit	GET	`/api/v1/audit-events`
Administration	GET	`/api/v1/admin/users`
Administration	GET	`/api/v1/admin/users/{user_id}`
Administration	POST	`/api/v1/admin/users/{user_id}/memberships`
Administration	PATCH, DELETE	`/api/v1/admin/users/{user_id}/memberships/{membership_id}`
Administration	PATCH	`/api/v1/admin/users/{user_id}/status`

Use **`/docs`** for request bodies, response schemas, authentication controls, and live Swagger execution.

8. Audit Events

Membership and account actions add **`AuditEvent`** records:

Action	Resource
`membership.assigned`	**`organisation_membership`**
`membership.updated`	**`organisation_membership`**
`membership.revoked`	**`organisation_membership`**
`user.status.updated`	**`user`**

Audit metadata contains IDs and changed values. Passwords, tokens, and plaintext credentials are not included.

9. Migration and Database Commands

No schema migration is needed for this update.

```
.venv\Scripts\alembic.exe current
.venv\Scripts\alembic.exe check
.venv\Scripts\alembic.exe upgrade head
```

Verified results:

• Alembic revision: **`phase4_platform_workflows`**

• Revision is the migration head

• **`alembic check`**: no new upgrade operations detected

• MySQL 8.0-backed application connection verified by the live server

Do not run a migration downgrade or database reset.

10. Test Results

Command:

```
.venv\Scripts\pytest.exe -q
```

Actual result:

• **`55 passed`**

• **`0 failed`**

• **`0 skipped`**

• Passes include authentication, registration, admin authorization, membership lifecycle, invalid roles, duplicate membership, account status, audit events, and the existing Phase 1–4 coverage

Focused registration/admin command:

```
.venv\Scripts\pytest.exe -q tests/test_auth.py tests/test_admin.py
```

Result:

• **`12 passed`**

• **`0 failed`**

• **`0 skipped`**

Tests use in-memory SQLite. MySQL migration/model compatibility was separately verified with Alembic.

11. Live Swagger and Server Verification

Start or restart the server:

```
.venv\Scripts\Activate.ps1
.venv\Scripts\uvicorn.exe app.main:app --host 127.0.0.1 --port 8000
```

Verified URLs:

• Swagger: **`http://127.0.0.1:8000/docs`**

• ReDoc: **`http://127.0.0.1:8000/redoc`**

• OpenAPI: **`http://127.0.0.1:8000/openapi.json`**

• Health: **`http://127.0.0.1:8000/api/v1/health`**

The restarted server returned HTTP 200 for Swagger, ReDoc, and health. Its OpenAPI document contains the registration route and all six requested administrative operations. An unauthenticated request to **`/api/v1/admin/users`** returned 401.

12. Known Limitations

• Automated tests use in-memory SQLite; they do not execute the MySQL migration SQL.

• Live administrative mutations were not executed because the initial administrator has no documented bootstrap and is intentionally provisioned manually.

• Public registration does not verify email ownership; **`is_verified=false`**.

• No invitation mechanism was created.

• Revocation is soft and prevents active membership authorization by existing role checks; the historical membership row remains.

• No cache or search index exists in the inspected implementation, so no cache invalidation mechanism requires verification.